

Lab Work Section 2

Overview

This lab work focuses on the core concepts of pointers in C, including declaration, dereferencing, pointer arithmetic, and multi-level pointers.

Part 1: Basic Pointer Declaration & Dereferencing (15 pts)

Problem 1: Integer Pointer Basics (5 pts)

- **Function Name:** `basicPointerOps`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare an integer variable `num` initialized to 10.
 2. Declare a pointer `ptr` and assign the address of `num` to it.
 3. Print the value of `num`.
 4. Print the value of `num` using `ptr`.
 5. Print the address of `num`.
 6. Print the address stored in `ptr`.
- **Example Output:**

```
Value: 10
Value via pointer: 10
Address: 0x7ffee4
Address in pointer: 0x7ffee4
```

Problem 2: Float Pointer Basics (5 pts)

- **Function Name:** `floatPointerOps`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare a float variable `pi` initialized to 3.14.
 2. Declare a float pointer `p_pi` pointing to `pi`.
 3. Print the value of `pi` directly and via `p_pi`.
 4. Print the memory address of `pi`.

Problem 3: Character Pointer Basics (5 pts)

- **Function Name:** `charPointerOps`
- **Return Type:** void
- **Parameters:** None
- **Description:**

1. Declare a char variable `letter` initialized to 'A'.
2. Declare a char pointer `p_letter` pointing to `letter`.
3. Print the character using the pointer.
4. Print the ASCII value of the character using the pointer.

Part 2: Updating Variables Using Pointers (15 pts)

Problem 4: Modifying Integer (5 pts)

- **Function Name:** `modifyScore`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare an integer `score = 0`.
 2. Declare a pointer `p_score` pointing to `score`.
 3. Update `score` to 50 using **only** `p_score`.
 4. Print `score` to verify.

Problem 5: Modifying Double (5 pts)

- **Function Name:** `updatePrice`
- **Return Type:** void
- **Parameters:** None
- **Description:**
 1. Declare a double `price = 9.99`.
 2. Create a pointer to it.
 3. Change the value to 19.99 using the pointer.
 4. Print the new value.

Problem 6: Conditional Update (5 pts)

- **Function Name:** `checkAndReset`
- **Return Type:** void
- **Parameters:** `int *p_val`
- **Description:** Check if the value pointed to by `p_val` is greater than 50. If so, set it to 0.
- **Example:**
 - Input: `val = 100`
 - Call: `checkAndReset(&val)`
 - Result: `val` becomes 0

Part 3: Pointer Arithmetic on Arrays (20 pts)

Problem 7: Array Traversal (7 pts)

- **Function Name:** `printArrayPtr`
- **Return Type:** void
- **Parameters:** `int *arr, int size`
- **Description:** Print all elements of the array using pointer arithmetic (incrementing the pointer). **Do not use array indexing `arr[i]`.**

Problem 8: Sum of Array (7 pts)

- **Function Name:** `sumArrayPtr`
- **Return Type:** `float`
- **Parameters:** `float *arr, int size`
- **Description:** Calculate and return the sum of all elements in the float array using pointer arithmetic.

Problem 9: Reverse Traversal (6 pts)

- **Function Name:** `printReversePtr`
- **Return Type:** `void`
- **Parameters:** `int *arr, int size`
- **Description:** Initialize a pointer to the last element of the array and iterate backwards to print elements in reverse order.

Part 4: Functions and Pointers (25 pts)

Problem 10: Basic Swap (8 pts)

- **Function Name:** `swap`
- **Return Type:** `void`
- **Parameters:** `int *a, int *b`
- **Description:** Swap the values of two integers pointed to by `a` and `b`.
- **Example:**
 - Input: `a=5, b=10`
 - Output: `a=10, b=5`

Problem 11: Double It (8 pts)

- **Function Name:** `doubleIt`
- **Return Type:** `void`
- **Parameters:** `int *n`
- **Description:** Multiply the value at address `n` by 2.

Problem 12: Division and Modulo (9 pts)

- **Function Name:** `div_mod`
- **Return Type:** `void`
- **Parameters:** `int a, int b, int *div, int *mod`
- **Description:** Calculate `a / b` and `a % b`, storing the results in the locations pointed to by `div` and `mod`.

Part 5: Double Pointers (25 pts)

Problem 13: Basic Double Pointer (8 pts)

- **Function Name:** `demonstrateDoublePtr`
- **Return Type:** `void`
- **Parameters:** None

- **Description:**
 1. Create an integer `x = 100`.
 2. Create a pointer `ptr` pointing to `x`.
 3. Create a double pointer `pptr` pointing to `ptr`.
 4. Print `x` using `ptr` and `pptr`.

Problem 14: Modification via Double Pointer (8 pts)

- **Function Name:** `modifyViaDoublePtr`
- **Return Type:** `void`
- **Parameters:** `int **pp`
- **Description:** Change the value of the integer pointed to by the pointer pointed to by `pp` to 200.

Problem 15: Pointer Reassignment (9 pts)

- **Function Name:** `pointToB`
- **Return Type:** `void`
- **Parameters:** `int **pp, int *newTarget`
- **Description:** Update the pointer pointed to by `pp` so that it points to `newTarget`.

Submission Guidelines

1. Submit a single `.c` file containing all implementations.
2. Include comments explaining your logic.
3. Test your code with a `main` function that calls all these functions.

Grading

- Correct implementation of Part 1: 15 points
- Correct implementation of Part 2: 15 points
- Correct implementation of Part 3: 20 points
- Correct implementation of Part 4: 25 points
- Correct implementation of Part 5: 25 points
- Code style and documentation: Additional consideration

Total: 100 points

Additional Notes

Bonus Points System

- Using bitwise operations where applicable: +5 points
- creating a menu-driven `main` function to test individual problems: +5 points

Good luck!